

tf.compat.v1.distributions.Normal Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/distributions/Normal

The Normal distribution with location loc and scale parameters.

Function Signatures

```
tf.compat.v1.distributions.Normal( loc, scale,
validate_args=False, allow_nan_stats=True, name='Normal' )
pdf(x; mu, sigma) = exp(-0.5 (x - mu)**2 / sigma**2) / Z Z =
(2 pi sigma**2)**0.5
```

Code Examples

```
tf.compat.v1.distributions.Normal(
loc,
scale,
validate_args=False,
allow_nan_stats=True,
name='Normal'
)
```

```
tf.compat.v1.distributions.Normal(  
    loc,  
    scale,  
    validate_args=False,  
    allow_nan_stats=True,  
    name='Normal'  
)
```

```
pdf(x; mu, sigma) = exp(-0.5 (x - mu)**2 / sigma**2) / Z  
Z = (2 pi sigma**2)**0.5
```

```
pdf(x; mu, sigma) = exp(-0.5 (x - mu)**2 / sigma**2) / Z  
Z = (2 pi sigma**2)**0.5
```

```
X ~ Normal(loc=0, scale=1)  
Y = loc + scale * X
```

```
X ~ Normal(loc=0, scale=1)  
Y = loc + scale * X
```

```
import tensorflow_probability as tfp
tfd = tfp.distributions
# Define a single scalar Normal distribution.
dist = tfd.Normal(loc=0., scale=3.)
# Evaluate the cdf at 1, returning a scalar.
dist.cdf(1.)
# Define a batch of two scalar valued Normals.
# The first has mean 1 and standard deviation 11, the second 2 and
22.
dist = tfd.Normal(loc=[1, 2.], scale=[11, 22.])
# Evaluate the pdf of the first distribution on 0, and the second
on 1.5,
# returning a length two tensor.
dist.prob([0, 1.5])
# Get 3 samples, returning a 3 x 2 tensor.
dist.sample([3])
```

```
import tensorflow_probability as tfp
tfd = tfp.distributions
# Define a single scalar Normal distribution.
dist = tfd.Normal(loc=0., scale=3.)
# Evaluate the cdf at 1, returning a scalar.
dist.cdf(1.)
# Define a batch of two scalar valued Normals.
# The first has mean 1 and standard deviation 11, the second 2 and
22.
dist = tfd.Normal(loc=[1, 2.], scale=[11, 22.])
# Evaluate the pdf of the first distribution on 0, and the second
on 1.5,
# returning a length two tensor.
dist.prob([0, 1.5])
# Get 3 samples, returning a 3 x 2 tensor.
dist.sample([3])
```

Documentation

The Normal distribution with location `loc` and scale parameters.

Inherits From: Distribution

Mathematical details

The probability density function (pdf) is,

where $\text{loc} = \mu$ is the mean, $\text{scale} = \sigma$ is the std. deviation, and, Z is the normalization constant.

The Normal distribution is a member of the location-scale family, i.e., it can be constructed as,

Examples

Examples of initialization of one or a batch of distributions.

Arguments are broadcast when possible.

Raises

Attributes

Stats return +/- infinity when it makes sense. E.g., the variance of a Cauchy distribution is infinity. However, sometimes the statistic is undefined, e.g., if a distribution's pdf does not achieve a maximum within the support of the distribution, the mode is undefined. If the mean is undefined, then by definition the variance is undefined. E.g. the mean for Student's T for $\text{df} = 1$ is undefined (no clear way to say it is either + or - infinity), so the variance $= E[(X - \text{mean})^2]$ is also undefined.

May be partially defined or unknown.

The batch dimensions are indexes into independent, non-identical parameterizations of this distribution.

May be partially defined or unknown.

Currently this is one of the static instances `distributions.FULLY_REPARAMETERIZED` or `distributions.NOT_REPARAMETERIZED`.

Methods

`## batch_shape_tensor`

[View source](#)

Shape of a single sample from a single event index as a 1-D Tensor.

The batch dimensions are indexes into independent, non-identical parameterizations of this distribution.

[View source](#)

Cumulative distribution function.

Given random variable X , the cumulative distribution function cdf is:

[View source](#)

Creates a deep copy of the distribution.

covariance

[View source](#)

Covariance.

Covariance is (possibly) defined only for non-scalar-event distributions.

For example, for a length- k , vector-valued distribution, it is calculated as,

where Cov is a (batch of) $k \times k$ matrix, $0 \leq (i, j) < k$, and E denotes expectation.

Alternatively, for non-vector, multivariate distributions (e.g., matrix-valued, Wishart), Covariance shall return a (batch of) matrices under some vectorization of the events, i.e.,

where Cov is a (batch of) $k' \times k'$ matrices, $0 \leq (i, j) < k' = \text{reduce_prod}(\text{event_shape})$, and Vec is some function mapping indices of this distribution's event dimensions to indices of a length- k' vector.

cross_entropy

[View source](#)

Computes the (Shannon) cross entropy.

Denote this distribution (self) by P and the other distribution by Q . Assuming P, Q are absolutely continuous with respect to one another and permit densities $p(x) \, dx$ and $q(x) \, dx$, (Shanon)

cross entropy is defined as:

where F denotes the support of the random variable $X \sim P$.

entropy

[View source](#)

Shannon entropy in nats.

event_shape_tensor

[View source](#)

Shape of a single sample from a single batch as a 1-D int32 Tensor.

is_scalar_batch

[View source](#)

Indicates that `batch_shape == []`.

is_scalar_event

[View source](#)

Indicates that `event_shape == []`.

kl_divergence

[View source](#)

Computes the Kullback--Leibler divergence.

Denote this distribution (self) by p and the other distribution by

q. Assuming p, q are absolutely continuous with respect to reference measure r , the KL divergence is defined as:

where F denotes the support of the random variable $X \sim p$, $H[., .]$ denotes (Shanon) cross entropy, and $H[.]$ denotes (Shanon) entropy.

log_cdf

[View source](#)

Log cumulative distribution function.

Given random variable X , the cumulative distribution function cdf is:

Often, a numerical approximation can be used for $\log_cdf(x)$ that yields a more accurate answer than simply taking the logarithm of the cdf when $x \ll -1$.

log_prob

[View source](#)

Log probability density/mass function.

log_survival_function

[View source](#)

Log survival function.

Given random variable X , the survival function is defined:

Typically, different numerical approximations can be used for the log survival function, which are more accurate than $1 - \text{cdf}(x)$ when $x \gg 1$.

[View source](#)

[View source](#)

param_shapes

[View source](#)

Shapes of parameters given the desired shape of a call to `sample()`.

This is a class method that describes what key/value arguments are required to instantiate the given Distribution so that a particular shape is returned for that instance's call to `sample()`.

Subclasses should override class method `_param_shapes`.

param_static_shapes

[View source](#)

`param_shapes` with static (i.e. `TensorShape`) shapes.

This is a class method that describes what key/value arguments are required to instantiate the given Distribution so that a particular shape is returned for that instance's call to `sample()`. Assumes that the sample's shape is known statically.

Subclasses should override class method `_param_shapes` to return constant-valued tensors when constant values are fed.

[View source](#)

Probability density/mass function.

quantile

[View source](#)

Quantile function. Aka "inverse cdf" or "percent point function".

Given random variable X and p in $[0, 1]$, the quantile is:

sample

[View source](#)

Generate samples of the specified shape.

Note that a call to `sample()` without arguments will generate a single sample.

stddev

[View source](#)

Standard deviation.

Standard deviation is defined as,

where X is the random variable associated with this distribution, E denotes expectation, and `stddev.shape = batch_shape + event_shape`.

survival_function

[View source](#)

Survival function.

Given random variable X , the survival function is defined:

variance

[View source](#)

Variance.

Variance is defined as,

where X is the random variable associated with this distribution, E denotes expectation, and $\text{Var.shape} = \text{batch_shape} + \text{event_shape}$.